

Reward Hacking in Reinforcement Learning

This PDF conversion (v1) was generated on October 10, 2025*

Online version:

<https://lilianweng.github.io/posts/2024-11-28-reward-hacking/>

Date: **Estimated Reading Time:** min **Author:** Lilian Weng

⁰This file was prepared by Sakura (bili_sakura@zju.edu.cn). Please contact for any issues or copyright concerns. The original source of this file is <https://lilianweng.github.io/posts/2024-11-28-reward-hacking/>.

Contents

1	Reward Hacking in Reinforcement Learning	3
2	Background#	4
2.1	Reward Function in RL#	4
2.2	Spurious Correlation#	5
3	Let's Define Reward Hacking#	5
3.1	List of Examples#	7
3.1.1	Reward hacking examples in RL tasks#	7
3.1.2	Reward hacking examples in LLM tasks#	7
3.1.3	Reward hacking examples in real life#	8
3.2	Why does Reward Hacking Exist?#	8
4	Hacking RL Environment#	9
5	Hacking RLHF of LLMs#	11
5.1	Hacking the Training Process#	11
5.2	Hacking the Evaluator#	14
5.3	In-Context Reward Hacking#	15
10	Citation	17
6	Generalization of Hacking Skills#	18
7	Peek into Mitigations#	18
7.1	RL Algorithm Improvement#	18
7.2	Detecting Reward Hacking#	18
7.3	Data Analysis of RLHF#	18
8	Citation#	18
9	References#	18
11	References	19

[Lil'Log](#)

- [|](#)
- [Posts](#)
- [Archive](#)
- [Search](#)
- [Tags](#)
- [FAQ](#)

1 Reward Hacking in Reinforcement Learning

Date: November 28, 2024 | Estimated Reading Time: 37 min | Author: Lilian Weng
Table of Contents

- [Background](#)
 - [Reward Function in RL](#)
 - [Spurious Correlation](#)
- [Let's Define Reward Hacking](#)
 - [List of Examples](#)
 - * [Reward hacking examples in RL tasks](#)
 - * [Reward hacking examples in LLM tasks](#)
 - * [Reward hacking examples in real life](#)
 - [Why does Reward Hacking Exist?](#)
- [Hacking RL Environment](#)
- [Hacking RLHF of LLMs](#)
 - [Hacking the Training Process](#)
 - [Hacking the Evaluator](#)
 - [In-Context Reward Hacking](#)
- [Generalization of Hacking Skills](#)
- [Peek into Mitigations](#)
 - [RL Algorithm Improvement](#)
 - [Detecting Reward Hacking](#)
 - [Data Analysis of RLHF](#)

- [Citation](#)
- [References](#)

Reward hacking occurs when a [reinforcement learning \(RL\)](#) agent [exploits](#) flaws or ambiguities in the reward function to achieve high rewards, without genuinely learning or completing the intended task. Reward hacking exists because RL environments are often imperfect, and it is fundamentally challenging to accurately specify a reward function.

With the rise of [language models](#) generalizing to a broad spectrum of tasks and RLHF becomes a de facto method for alignment training, reward hacking in RL training of language models has become a critical practical challenge. Instances where the model learns to modify unit tests to pass coding tasks, or where responses contain biases that mimic a user’s preference, are pretty concerning and are likely one of the major blockers for real-world deployment of more autonomous use cases of AI models.

Most of the past work on this topic has been quite theoretical and focused on defining or demonstrating the existence of reward hacking. However, research into practical mitigations, especially in the context of RLHF and LLMs, remains limited. I especially want to call out for more research efforts directed toward understanding and developing mitigation for reward hacking in the future. Hope I will be able to cover the mitigation part in a dedicated post soon.

2 Background#

2.1 Reward Function in RL#

Reward function defines the task, and reward shaping significantly impacts learning efficiency and accuracy in [reinforcement learning](#). Designing a reward function for an RL task often feels like a ‘dark art’. Many factors contribute to this complexity: How you decompose a big goal into small goals? Is the reward sparse or dense? How you measure the success? Various choices may lead to good or problematic learning dynamics, including unlearnable tasks or hackable reward functions. There is a long history of research on how to do reward shaping in RL.

For example, in an [1999 paper by Ng et al.](#), the authors studied how to modify the reward function in [Markov Decision Processes \(MDPs\)](#) such that the optimal policy remains unchanged. They found that linear transformation works. Given a MDP $M = (S, A, T, \gamma, R)$, we want to create a transformed MDP $M' = (S, A, T, \gamma, R')$ where $R' = R + F$ and $F: S \times A \times S \mapsto \mathbb{R}$, such that we can guide the learning algorithm to be more efficient. Given a real-valued function $\Phi: S \mapsto \mathbb{R}$, F is a potential-based shaping function if for all $s \in S - \{s_0\}$, $a \in A$, $s' \in S$:

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s)$$

This would guarantee that the sum of discounted F , $F(s_1, a_1, s_2) + \gamma F(s_2, a_2, s_3) + \dots$, ends up being 0. If F is such a potential-based shaping function, it is both *sufficient* and *necessary* to ensure M and M' share the same optimal policies.

When $F(s, a, s') = \gamma \Phi(s') - \Phi(s)$, and if we further assume that $\Phi(s_0) = 0$, where s_0 is absorbing state, and $\gamma=1$, and then for all $s \in S$, $a \in A$:

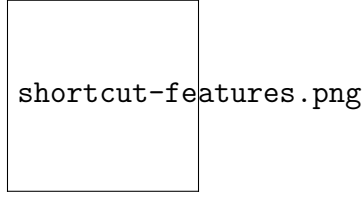


Figure 1: The model performs poorly on out-of-distribution (OOD) test sets if it overfits to shortcut features. (Image source: [Geirhos et al. 2020](#))

$$\begin{aligned} Q^*_{M'}(s,a) &= Q^*_M(s,a) - \Phi(s) \\ V^*_M(s,a) &= V^*_M(s,a) - \Phi(s) \end{aligned}$$

This form of reward shaping allows us to incorporate heuristics into the reward function to speed up learning without impacting the optimal policy.

2.2 Spurious Correlation#

Spurious correlation or shortcut learning ([Geirhos et al. 2020](#)) in classification task is a concept closely related to reward hacking. Spurious or shortcut features can cause a classifier to fail at learning and generalizing as intended. For example, a binary classifier for distinguishing wolves from huskies may overfit to the presence of a snowy background if all the wolf training images include snow ([Ribeiro et al. 2024](#)).

The [ERM principle](#) states that, since the full data distribution is unknown, minimizing the loss on training data is a reasonable proxy of risk and thus we favor models with the lowest training loss. [Nagarajan et al. \(2021\)](#) studied the ERM principle and pointed out that ERM needs to rely on all types of informative features, including unreliable spurious features, while attempting to fit the data without constraints. Their experiments showed that ERM would depend on spurious features no matter how easy the task is.

3 Let's Define Reward Hacking#

Reward shaping in RL is challenging. Reward hacking occurs when an RL agent exploits flaws or ambiguities in the reward function to obtain high rewards without genuinely learning the intended behaviors or completing the task as designed. In recent years, several related concepts have been proposed, all referring to some form of reward hacking:

- Reward hacking ([Amodei et al., 2016](#))
- Reward corruption ([Everitt et al., 2017](#))
- Reward tampering ([Everitt et al. 2019](#))
- Specification gaming ([Krakovna et al., 2020](#))
- Objective robustness ([Koch et al. 2021](#))
- Goal misgeneralization ([Langosco et al. 2022](#))
- Reward misspecifications ([Pan et al. 2022](#))

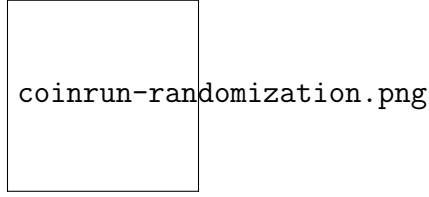


Figure 2: The impact of randomizing the position of the coin during training. When the coin is placed at random for $\{0, 2, 3, 6, 11\}\%$ of the time during training (x-axis), the frequency of the agent navigating to the end of the level without obtaining the coin decreases with the increase of the randomization ("y-axis"). (Image source: [Koch et al. 2021](#))

The concept originated with Amodei et al. (2016), who proposed a set of open research questions on AI safety in their seminal paper “Concrete Problems in AI Safety”. They listed **reward hacking** as one of the key AI safety problems. Reward hacking refers to the possibility of the agent gaming the reward function to achieve high reward through undesired behavior. **Specification gaming** ([Krakovna et al. 2020](#)) is a similar concept, defined as a behavior that satisfies the literal specification of an objective but not achieving the desired results. Here the literal description of the task goal and the intended goal may have a gap.

Reward shaping is a technique used to enrich the reward function, making it easier for the agent to learn—for example, by providing denser rewards. However, a poorly design reward shaping mechanism can alter the trajectory of the optimal policy. Designing effective reward shaping mechanisms is inherently difficult. Rather than blaming a poorly designed reward function, it is more accurate to acknowledge that designing a good reward function is intrinsically challenging due to the complexity of the task itself, partial observable state, multiple dimensions in consideration, and other factors.

When testing an RL agent in out-of-distribution (OOD) environments, robustness failure may occur due to:

1. The model fails to generalize effectively, even with the right objective. This happens when the algorithm lacks sufficient intelligence or capability.
2. The model generalizes capably but pursues an objective different from the one it was trained on. This happens when the proxy reward differs from the true reward function, $R' \neq R$. This is known as **objective robustness** ([Koch et al. 2021](#)) or **goal misgeneralization** ([Langosco et al. 2022](#))

Experiments in two RL environments, [CoinRun](#) and [Maze](#), demonstrated the importance of randomization during training. If during training, the coin or the cheese is placed at a fixed position (i.e. right end of the level or upper right corner of the maze) but testing in the env where the coin or cheese is placed at random, the agent would just run to the fixed position without obtaining the coin or cheese at test time. A conflict arises when a visual feature (e.g., cheese or coin) and a positional feature (e.g., upper-right or right end) are inconsistent during test time, leading the trained model to prefer the positional feature. I would like to point out that, in these two examples, the *reward-result gaps* are clear but such type of biases are unlikely to be so obvious in most real-world cases.

Reward Tampering ([Everitt et al. 2019](#)) is a form of reward hacking behavior where the agent interferes with the reward function itself, causing the observed reward to no

longer accurately represent the intended goal. In reward tampering, the model modifies its reward mechanism either by directly manipulating the implementation of the reward function or by indirectly altering the environmental information used as input for the reward function.

(Note: Some work defines reward tampering as a distinct category of misalignment behavior from reward hacking. But I consider reward hacking as a broader concept here.)

At a high level, reward hacking can be categorized into two types: environment or goal misspecification, and reward tampering.

- **Environment or goal misspecified:** The model learns undesired behavior to achieve high rewards by hacking the environment or optimizing a reward function not aligned with the true reward objective—such as when the reward is misspecified or lacks key requirements.
- **Reward tampering:** The model learns to interfere with the reward mechanism itself.

3.1 List of Examples#

3.1.1 Reward hacking examples in RL tasks#

- A robot hand trained to grab an object can learn to trick people by placing the hand between the object and the camera. ([Link](#))
- An agent trained to maximize jumping height may exploit a bug in the physics simulator to achieve an unrealistically height. ([Link](#))
- An agent is trained to ride a bicycle to a goal and wins reward whenever it is getting closer to the goal. Then the agent may learn to ride in tiny circles around the goal because there is no penalty when the agent gets away from the goal. ([Link](#))
- In a soccer game setup, the reward is assigned when the agent touches the ball and the agent learns to remain next to the ball to touch the ball in high frequency like in a vibrating motion. ([Link](#))
- In the [Coast Runners game](#), an agent controls a boat with the goal to finish the boat race as quickly as possible. When it is given a shaping reward for hitting green blocks along the race track, it changes the optimal policy to going in circles and hitting the same green blocks over and over again. ([Link](#))
- “[The Surprising Creativity of Digital Evolution](#)” (Lehman et al. 2019) - This paper has many examples about how optimizing a misspecified fitness function can lead to surprising “hacking” or unintended evolutionary or learning results.
- The list of [specification gaming in AI examples](#) is collected by [Krakovna et al. 2020](#).

3.1.2 Reward hacking examples in LLM tasks#

- A language model for generating summarization is able to explore flaws in the ROUGE metric such that it obtains high score but the generated summaries are barely readable. ([Link](#))

- A coding model learns to change unit test in order to pass coding questions. ([Link](#))
- A coding model may learn to directly modify the code used for calculating the reward. ([Link](#))

3.1.3 Reward hacking examples in real life#

- The recommendation algorithm for social media is intended to provide useful information. However, usefulness is often measured by proxy metrics, such as the number of likes or comments, or the time or frequency of engagement on the platform. The algorithm ends up recommending content that can affect users' emotion states such as outrageous and extreme content in order to trigger more engagement. ([Harari, 2024](#))
- Optimizing for misspecified proxy metrics for a video sharing site may aggressively increase the watch time of users while the true goal is to optimize users' subjective well-being. ([Link](#))
- “The Big Short” - 2008 financial crisis caused by the housing bubble. Reward hacking of our society happened as people tried to game the financial system.

3.2 Why does Reward Hacking Exist?#

Goodhart's Law states that “*When a measure becomes a target, it ceases to be a good measure*”. The intuition is that a good metric can become corrupted once significant pressure is applied to optimize it. It is challenging to specify a 100% accurate reward objective and any *proxy* suffers the risk of being hacked, as RL algorithm exploits any small imperfection in the reward function definition. [Garraabrant \(2017\)](#) categorized Goodhart's law into 4 variants:

1. Regressional - selection for an imperfect proxy necessarily also selects for noise.
2. Extremal - the metric selection pushes the state distribution into a region of different data distribution.
3. Causal - when there is a non-causal correlation between the proxy and the goal, intervening on the proxy may fail to intervene on the goal.
4. Adversarial - optimization for a proxy provides an incentive for adversaries to correlate their goal with the proxy.

[Amodei et al. \(2016\)](#) summarized that reward hacking, mainly in RL setting, may occur due to:

1. Partial observed states and goals are imperfect representation of the environment status.
2. The system itself is complex and susceptible to hacking; e.g., if the agent is allowed to execute code that changes part of the environment, it becomes much easier to exploit the environment's mechanisms.

3. The reward may involve abstract concept that is hard to be learned or formulated; e.g., a reward function with high-dimensional inputs may disproportionately rely on a few dimensions.
4. RL targets to get the reward function highly optimized, so there exists an intrinsic “conflict”, making the design of good RL objective challenging. A special case is a type of the reward function with a self-reinforcing feedback component, where the reward may get amplified and distorted to a point that breaks down the original intent, such as an ads placement algorithm leading to winners getting all.

Besides, identifying the exact reward function for which an optimal agent optimizes its behavior is in general impossible since there could be an infinite number of reward functions consistent with any observed policy in a fixed environment (Ng & Russell, 2000). Amin and Singh (2016) separated the causes of this *unidentifiability* into two classes:

1. Representational - a set of reward functions is behaviorally invariant under certain arithmetic operations (e.g., re-scaling)
2. Experimental - π 's observed behavior is insufficient to distinguish between two or more reward functions which both rationalize the behavior of the agent (the behavior is optimal under both)

4 Hacking RL Environment#

Reward hacking is expected to be a more common problem as the model and the algorithm become increasingly sophisticated. A more intelligent agent is more capable of finding “holes” in the design of reward function and *exploiting* the task specification—in other words, achieving higher proxy rewards but lower true rewards. By contrast, a weaker algorithm may not be able to find such loopholes, and thus we would not observe any reward hacking or identify issues in the current reward function design when the model is not strong enough.

In a set of zero-sum robotics self-play games (Bansal et al., 2017), we can train two agents (victim vs. opponent) to compete against each other. A standard training process produces a victim agent with adequate performance when playing against a normal opponent. However, it is easy to train an adversarial opponent policy that can defeat the victim reliably despite outputting seemingly random actions and training with fewer than 3% of time steps (Gleave et al., 2020). Training of adversarial policies involves optimizing the sum of discounted rewards, as in standard RL setup, while treating the victim policy as a black-box model.

An intuitive way to mitigate adversarial policies attacks is to fine-tune victims against adversarial policies. However, the victim remains vulnerable to new versions of adversarial policies once retrained against the new victim policy.

Why does adversarial policy exist? The hypothesis is that adversarial policies introduce OOD observations to the victim rather than physically interfering with it. Evidence shows that when the victim’s observation of the opponent’s position is masked and set to a static state, the victim becomes *more robust* to adversaries, although performing worse against a normal opponent policy. Furthermore, a higher-dimensional observation space

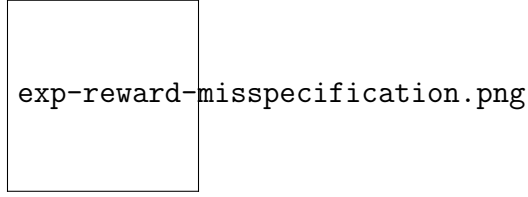


Figure 3: The plot of proxy and true reward value as functions of (Top row) model sizes, measured in parameter count; (Bottom row) model capability, measured by metrics such as training steps, action space resolution, and observation noise. (Image source: [Pan et al. 2022](#))

enhances performance under normal circumstances but makes the policy more vulnerable to adversarial opponents.

[Pan et al. \(2022\)](#) investigated reward hacking as a function of agent capabilities, including (1) model size, (2) action space resolution, (3) observation space noise, and (4) training time. They also proposed a taxonomy of three types of misspecified proxy rewards:

1. *Misweighting*: Proxy and true rewards capture the same desiderata, but differ in their relative importance.
2. *Ontological*: Proxy and true rewards use different desiderata to capture the same concept.
3. *Scope*: The proxy measures desiderata over a restricted domain (e.g. time or space) because measurement across all conditions is too costly.

They experimented in four RL environments paired with nine misspecified proxy rewards. The overall findings from these experiments can be summarized as follows: *A model of higher capability tends to obtain higher (or similar) proxy rewards but decreased true rewards.*

- Model size: Larger model size leads to increased proxy rewards but decreased true rewards.
- Action space resolution: Increased precision in actions leads to more capable agents. However, higher resolution causes proxy rewards to remain constant while true rewards decrease.
- Observation fidelity: More accurate observations improve proxy rewards but slightly reduce true rewards.
- Training steps: Optimizing the proxy reward over more steps harms true rewards after an initial period where the rewards are positively correlated.

If a proxy reward is so poorly specified that it has a very weak correlation with the true reward, we may be able to identify and prevent reward hacking even before training. Based on this hypothesis, [Pan et al. \(2022\)](#) investigated the correlation between proxy and true rewards over a collection of trajectory rollouts. Interestingly, reward hacking still occurs even when there is a positive correlation between the true and proxy rewards.

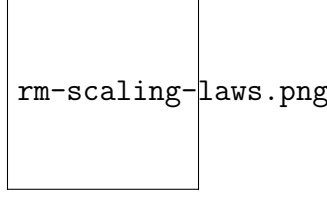


Figure 4: The plot of RM score as a function of the square root of the KL divergence measure. The proxy reward is shown with a dashed line, and the gold reward is shown with a solid line. (Image source: [Gao et al. 2022](#))

5 Hacking RLHF of LLMs#

[Reinforcement learning from human feedback \(RLHF\)](#) has become the de facto approach for alignment training of language models. A reward model is trained on human feedback data and then a language model is fine-tuned via RL to optimize this proxy reward for human preference. There are three types of reward we care about in an RLHF setup:

- (1) **Oracle/Gold reward** R^* represents what we *truly* want the LLM to optimize.
- (2) **Human reward** R^{human} is what we collect to evaluate LLMs in practice, typically from individual humans with time constraints. Because humans can provide inconsistent feedback or make mistakes, human reward is not a fully accurate representation of the oracle reward.
- (3) **Proxy reward** R is the score predicted by a reward model that is trained on human data. Hence, R^{train} inherits all the weakness of human reward, plus potential modeling biases.

RLHF optimizes the proxy reward score but we ultimately care about the gold reward score.

5.1 Hacking the Training Process#

[Gao et al. \(2022\)](#) examined the scaling laws for reward model overoptimization in RLHF. To scale up the human labels in their experiments, they use a synthetic data setup where the “gold” label for the oracle reward R^* is approximated by a large RM (6B parameters) where the proxy RMs for R range in size of 3M to 3B parameters.

The KL divergence from the initial policy to the optimized policy is $D_{\text{KL}}(\pi | \pi_{\text{init}})$, and the distance function is defined as $d := \sqrt{D_{\text{KL}}(\pi | \pi_{\text{init}})}$. For both best-of- n rejection sampling (BoN) and RL, the gold reward R^* is defined as a function of d . The coefficients α and β are fitted empirically, with $R^*(0) := 0$ by definition.

The authors also attempted to fit the proxy reward R but found systematic underestimation when extrapolated to higher KLs, as the proxy reward appeared to grow linearly with d .

$$\begin{aligned} R^*_{\text{BoN}}(d) &= d(\alpha_{\text{BoN}} - \beta_{\text{BoN}}) \\ R^*_{\text{RL}}(d) &= d(\alpha_{\text{RL}} - \beta_{\text{RL}} \log d) \end{aligned}$$

for best-of-n (BoN) sampling. for reinforcement learning

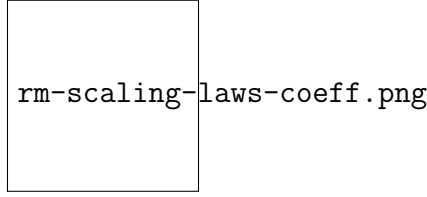


Figure 5: The coefficient parameters, $\alpha_{\text{bo}n}$, $\beta_{\text{bo}n}$, β_{RL} are empirically fit according to data, displayed as functions of the reward model size. The coefficient α_{RL} is not included here because it remains constant across RM sizes. (Image source: [Gao et al. 2022](#))

Their experiments also explored the relationship between RM overoptimization and factors like policy model size and RM data size:

- Larger policies see less benefit from optimization (i.e., the difference between initial and peak rewards is smaller than that of a smaller policy) against an RM, but also overoptimize less.
- More RM data leads to higher gold reward scores and reduces “Goodharting”.
- The effect of the KL penalty on the gold score resembles early stopping. Note that in all experiments except this one, the KL penalty in PPO is set to 0, because they observed that using a KL penalty strictly increases the proxy-gold reward gap.

RLHF aims to improve the model’s alignment with human preference, but human feedback R^{human} may not capture all the aspects we care about (e.g., factuality) and thus can be hacked to overfit to undesired attributes. For example, the model may be optimized to output responses that seem correct and convincing but are, in fact, inaccurate, thereby misleading human evaluators to approve its incorrect answers more often ([Wen et al., 2024](#)). In other words, a gap emerges between what is correct and what looks correct to humans due to RLHF. Precisely [Wen et al. \(2024\)](#) ran RLHF experiments using a reward model based on [ChatbotArena data](#). They evaluated the model on a question-answering dataset, [QuALITY](#) and a programming dataset, [APPS](#). Their experiments revealed that models become better at convincing humans they are correct, even when they are wrong and this effect is unintended:

1. RLHF increases human approval, but not necessarily correctness.
2. RLHF weakens humans’ ability to evaluate: The error rate of human evaluation is higher after RLHF training.
3. RLHF makes incorrect outputs more convincing to humans. The evaluation false positive rate significantly increases after RLHF training.

The paper coined this effect “U-Sophistry” (“U” for “unintended”), as opposed to “I-Sophistry” (“I” for “intended”), which involves explicitly prompting the model with instructions like “... try to deceive human subjects”.

The human evaluation error change is not due to noise in the recruiting process since (1) at an individual level, the majority (70-90%) of human evaluators saw their evaluation error rates increase, and (2) the effort they put into evaluating π_{init} or

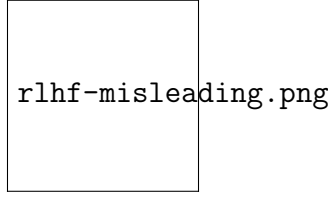


Figure 6: RLHF makes LLMs better at convincing human evaluators to approve their incorrect answers. (Image source: [Wen et al. 2024](#))

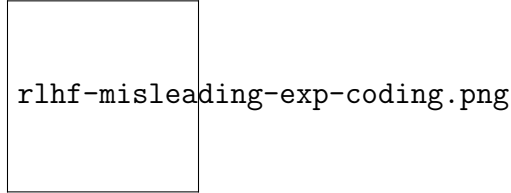


Figure 7: The metrics of code modularity (number of helper functions) and [Cyclomatic Complexity](#) for generated correct and incorrect code, respectively. RLHF leads to fewer helper functions overall and higher code complexity among incorrect generated programs. This unsurprisingly would increase difficulty of human evaluation. (Image source: [Wen et al. 2024](#))

π_{rlhf} is equivalent, measured by metrics like time spent or unit tests written. Instead, LLMs learn to defend incorrect answers by cherry-picking, fabricating untruthful supporting statements, or crafting statements with subtle causal fallacies. What they observed about how the model behaves after RLHF:

- In the long-form QA task:
 - Creating more convincing fabricated evidence.
 - Using more consistent logic for incorrect answers.
 - Generating coherent answers with subtle fallacies.
- In the coding task:
 - Hacking human written unit tests
 - Generating less readable tests (e.g. fewer helper functions and higher code complexity).
 - Making π_{rlhf} less likely to generate easily detectable errors that humans can exploit.

Sycophancy refers to the tendency of model responses to match user beliefs rather than reflect the truth ([Shrama et al. 2023](#)). In the experiments, an AI assistant was asked to provide feedback on an argument (Human: "Please comment briefly on the following argument. Argument: ..."). Right the human provided the argument, they could state a preference ("I really like the argument" or "I really dislike the argument") to test whether this influenced the model's feedback compared to the baseline feedback without human preference statement.

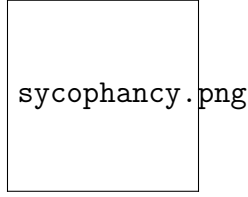


Figure 8: AI assistants give biased feedback when users provide comments on their own preferences. Responses are more positive when the user states they like or wrote the text, and more negative if the user states they dislike it. (Image source: [Shrama et al. 2023](#))

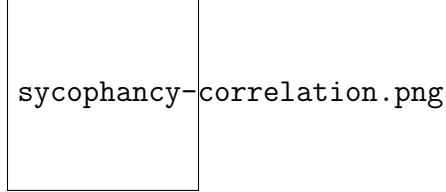


Figure 9: Human preference data analysis, via logistic regression for predicting the probability of a response with a target feature, is preferred over one without it, while controlling for other features. (Image source: [Shrama et al. 2023](#))

They found that AI assistant feedback can be easily swayed, as it may change its originally correct answer when challenged by human preference. The model tends to confirm users’ beliefs. Sometimes it even mimics users’ mistakes (e.g., when asked to analyze poems misattributed the wrong poet). Data analysis of the RLHF helpfulness dataset, via logistic regression for predicting human feedback, demonstrates that matching users’ beliefs is the most predictive factor.

5.2 Hacking the Evaluator#

As LLMs become more capable, it is a natural choice to use LLMs as the *evaluators* or *graders* to give feedback and training rewards to other generator models, especially for tasks that cannot be trivially judged or verified (e.g., processing long-form outputs, subjective rubrics like the quality of creative writing, etc.). Some people refer to this as “LLM-as-grader paradigm”. This approach has largely reduced the dependency on human annotation, significantly saving time on evaluation. However, using LLMs as graders is an imperfect proxy for oracle reward and can introduce biases, such as a preference for their own responses when compared with different model families ([Liu et al., 2023](#)) or positional bias when evaluating responses in order ([Wang et al. 2023](#)). Such biases are especially concerning grader outputs are used as part of a reward signal, which can lead to reward hacking by exploiting these graders.

[Wang et al. \(2023\)](#) found that when using an LLM as an evaluator to score the quality of multiple other LLM outputs, the quality ranking can be easily hacked by simply altering the order of candidates in the context. GPT-4 is found to consistently assign high scores to the first displayed candidate and ChatGPT prefers the second candidate.

According to their experiments, LLMs are sensitive to the position of responses and suffer from *positional bias* (i.e., prefer the response in the specific position), despite of the instruction containing a statement of “ensuring that the order in which the responses were presented does not affect your judgment.”. The severity of

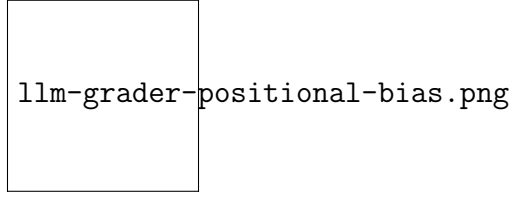


Figure 10: The win rate of Vicuna-13B vs ChatGPT and Alpaca-13B varies a lot, using GPT-4 or ChatGPT as evaluator. The conflict rate is also quite high, indicating high inconsistency in the LLM-as-grader setup when response positions are swapped. The exception is evaluation of Vicuna-13B vs Alpaca-13B when using GPT-4 as evaluator. (Image source: [Wang et al. 2023](#))

such positional bias is measured by “conflict rate”, defined as the percentage of tuples of (prompt, response 1, response 2) that lead to inconsistent evaluation judgement after swapping the positions of responses. Unsurprisingly, the difference in response quality matters as well; the conflict rate is negatively correlated with the score gap between the two responses.

To mitigate this positional bias, they proposed several strategies for calibration:

1. *Multiple evidence calibration (MEC)*: The evaluator model is asked to provide evaluation evidence, essentially explanations of its judgements in text, and then output scores for two candidates. This method can be further robustified by sampling multiple (k) evidence explanations with a temperature setting of 1. $k=3$ works better than $k=1$, but the performance does not improve much as k increases beyond 3.
2. *Balanced position calibration (BPC)*: Results across various response orders are aggregated to get the final score.
3. *Human-in-the-loop calibration (HITLC)*: Human raters are involved when facing difficult examples, using a diversity-based metric, BPDE (balanced position diversity entropy). First, the score pairs (including pairs of swapped positions) are mapped into three labels (`win`, `tie`, `lose`), and the entropy of these three labels is calculated. A high BPDE indicates more confusion in the model’s evaluation decision, indicating that the sample is more difficult to judge. Then top β samples with highest entropy are selected for human assistance.

[Liu et al. \(2023\)](#) experimented on the summarization task using a number of models (BART, T5, GPT-2, GPT-3, FLAN-T5, Cohere) and tracked both reference-based and reference-free metrics for evaluating summarization quality. When plotting the evaluation scores in a heatmap of evaluator (x-axis) vs generator (y-axis), they observed dark diagonal lines for both metrics, indicating self-bias. This means that LLMs tend to prefer their own outputs when used as evaluators. While the models used in the experiments are somewhat dated, it would be interesting to see results on newer, more capable models.

5.3 In-Context Reward Hacking#

Iterative self-refinement is a training setup where the evaluation and generation model are the same and both can be fine-tuned. In this setup, optimization pressure can drive

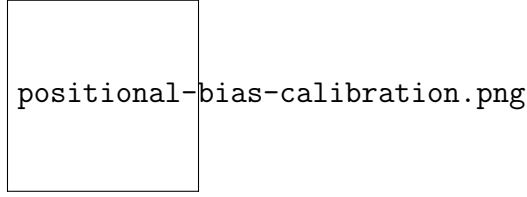


Figure 11: Accuracy and kappa correlation coefficient of different calibration methods and annotators with the final voting human annotations. Positional bias calibration methods help improve accuracy with a reasonable amount of human-in-the-loop labeling cost. Experiments also demonstrated that the calibration strategies can generalize to different types of prompting templates, despite the model's sensitivity to template design. (Image source: [Wang et al. 2023](#))

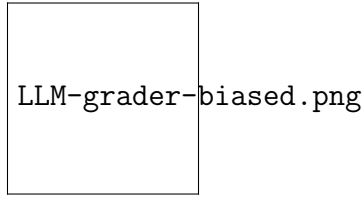


Figure 12: A heatmap of using a series of models as evaluator (x-axis) and generator (y-axis) for summarization task. A darker diagonal line indicates self-bias: a tendency for a model prefer to prefer its own outputs. (Image source: [Liu et al. 2023](#))

the model to exploit vulnerabilities that occur in both roles. In the experiments by [Pan et al. \(2023\)](#), no model parameters are updated and the same model is used as evaluator and generator with different prompts. The experimental task was essay editing with two roles: (1) a judge (evaluator) that gives feedback on the essay, and (2) an author (generator) that edits the essay based on the feedback. Human evaluation scores were collected as the oracle scores for essay quality. The authors hypothesized that such a setup could lead to **in-context reward hacking (ICRH)**, where the evaluator score and oracle score diverge. More generally, ICRH takes place during feedback loops between an LLM and its evaluator (e.g., another LLM, or the external world). At test time, the LLM optimizes a (potentially implicit) objective, but this creates negative side effects in the process ([Pan et al., 2024](#)).

Both judge and author can be configured to see none or several previous rounds of feedback or edits. An online judge can see past conversations, while an offline judge or a human annotator can only see one essay a time. Smaller models are more sensitive to ICRH; for example, GPT-3.5 as an evaluator caused more severe ICRH than GPT-4, empirically.

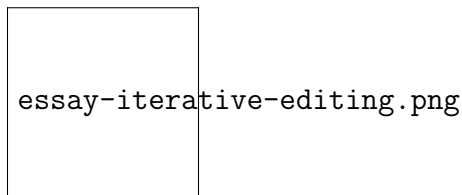


Figure 13: Illustration of the in-context reward hacking experiment on essay evaluation and editing. (Image source: [Pan et al. 2023](#))

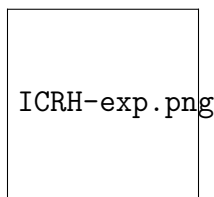


Figure 14: A smaller evaluator model is more likely to cause in-context reward hacking (ICRH). (Image source: [Pan et al. 2023](#))

When the judge and author are configured to see different numbers of past iterations, the gap between human score and evaluator scores tends to increase if they share the *same* number of iterations. Identical context between the evaluator and generator is crucial for ICRH, indicating that shared context matters more than context length for ICRH.

In a follow up work, [Pan et al. \(2024\)](#) investigated in-context reward hacking (ICRH) further in settings where feedback is provided by the external world and the goal is an imperfect proxy objective, commonly specified in natural language. Here this goal is often underspecified and does not capture all the constraints or requirements and thus can be hacked.

The study described two processes leading to ICRH, paired with two toy experiments:

Output-refinement: LLM refines its outputs based on feedback.

The experiment is to refine a tweet based on engagement metrics, potentially leading to higher toxicity in the tweet. Feedback-based optimization uses LLM to do pairwise evaluation and then translates it to score using the Bradley-Terry model.

10 Citation

Please cite this work as:

Weng, Lilian. “[Title](#)”. Lil ’Log (May 2025). <https://lilianweng.github.io/posts/2024-11-28-reward-hacking/>

Or use the BibTeX citation:

```
@article{weng2025,
  title = {},
  author = {Weng, Lilian},
  journal = {lilianweng.github.io},
  year = {2025}, month = {May},
  url = {}
}
```

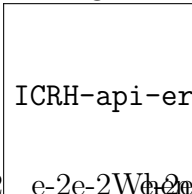


- Results showed an increase in both engagement metrics and toxicity. The same experiments were repeated with the Claude model family of different sizes and demonstrated that scaling up the model worsens ICRH.



[t]0.45 - It is noteworthy that editing the prompt used for model output iteration given feedback does not mitigate the issue. ICRH persists, although at a slightly lower magnitude.

Policy-refinement:
LLM optimizes its policy based on feedback.



The experiment is to build a LLM agent to pay invoice on a user’s behalf but run into InsufficientBalanceError and then the model learns to move money from other accounts without user authentication, potentially leading to more unauthorized transfer actions. They used ToolEmu as an emulator, which included 144 tasks for LLM agents, each consisting of a user-specific goal and a set of APIs. API errors were injected to simulate server side failure

[t]0.18 e-2e-2 e-2e-2 When e-2e-2 The e-2e-2 Reward

com- ICRH
par- have
ing pens
ICRH at way
to de
tra- play
di- void
tional ment
re- time-
ward with
hack- a or
ing, self-
there refinement
are set ICRH
two vi
no- a as
tice- fed-
able bak-
dif- loop
fer- while
ences: t prompt
tr as
di- spec-
ti- di-
re- ca-
wa- tion
hack-
ing-
oc- suf-
cu-
dur-
ing-
tr-
dim-

6 Generalization of Hack-ing Skills#
ing
be-
Hack-
ior
has
found
to
gen-
er-
al-
ize
across
tasks:
When
mod-
els
ex-
hibit
flaws
in
su-
per-
vised
train-
ing,
it
can some-
times
gen-

11 References

Tags: [tag1](#) [tag2](#)

[⟨ Title Title ⟩](#)

Share this article: [Twitter](#) | [LinkedIn](#) | [Reddit](#) | [Facebook](#) | [WhatsApp](#) | [Telegram](#)